2016 Paper E2.1: Digital Electronics II- Solutions

Q1 (a) This question tests students' understand the basic of FPGA and Verilog. They have already encountered the simple hex-to-7segment decoder in the lecture and the lab. This is an extension to that.

(i) This solution assumes that the tenth display is blank instead of showing a 0 when in ≤ 9:

```
module hex_to_BCD (dgt_1, dgt_0, in);
// this solution assumes tenth digits are blank or shows 1 if in>9
   output [6:0]   dgt_1, dgt_0;  // two display digits
   input [3:0]    in;            // binary inputs

   reg [6:0]      dgt_1, dgt_0;

   assign {dgt_1[6:3],dgt_1[0]} = 5'b11111; // all these segments are off
   always @(in)
      case (in)
      4'h0: {dgt_1[2:1], dgt_0}= 9'b111000000;
      4'h1: {dgt_1[2:1], dgt_0}= 9'b111111001;
      4'h2: {dgt_1[2:1], dgt_0}= 9'b110100100;
      4'h3: {dgt_1[2:1], dgt_0}= 9'b110110000;
      4'h4: {dgt_1[2:1], dgt_0}= 9'b110011001;
      4'h5: {dgt_1[2:1], dgt_0}= 9'b110010010;
      4'h6: {dgt_1[2:1], dgt_0}= 9'b110000010;
      4'h7: {dgt_1[2:1], dgt_0}= 9'b111111000;
      4'h8: {dgt_1[2:1], dgt_0}= 9'b110000000;
      4'h9: {dgt_1[2:1], dgt_0}= 9'b110010000;
      4'ha: {dgt_1[2:1], dgt_0}= 9'b001000000;
      4'hb: {dgt_1[2:1], dgt_0}= 9'b001111001;
      4'hc: {dgt_1[2:1], dgt_0}= 9'b000100100;
      4'hd: {dgt_1[2:1], dgt_0}= 9'b000110000;
      4'he: {dgt_1[2:1], dgt_0}= 9'b000011001;
      4'hf: {dgt_1[2:1], dgt_0}= 9'b000010010;
      endcase
endmodule
```

[6]

(ii) Since there are only 9 outputs that are dependent on 4 inputs, we only need 9 LEs for this decoder.

[2]

Q1 (b) This question tests students' understanding of the timing constraints in digital circuits.

(i)

Ignoring any hold time violation, the path from Q1 (FF1 Q output) to D2 governs the worst-case timing. This gives:

$$t_{clk} \geq 2 * t_{d(max)} + t_{cq(max)} + t_{su}$$

Therefore $t_{clk} \geq 2 * 1ns + 1ns + 1.3ns = 4.3ns$

Hence $f_{clk}(max) = 232.6MHz.$

[3]

(ii)

The shortest delay from Q2 (FF2 Q output) to D1 (FF1 D input) is:

$$t_{cq(min)} + t_{d(min)} = 1.0 \text{ ns}$$

which is shorter than the hold time requirement of FF1, i.e. 1.1ns. Therefore there is hold time violation at FF1 D input.

[2]

(iii)

To eliminate the hold time violation, we want to delay the arrival time at D1 by at least 0.1ns. This can be achieved by delaying to clock to FF2 by the same amount of 0.1ns. Therefore the minimum value required for t_{dly} is 0.1ns.

[2]

Now for FF2:

$$t_{clk} \geq 2 * t_{d(max)} + t_{cq(max)} + t_{su} - t_{dly}$$

Now for FF1:

$$t_{clk} \geq t_{d(max)} + t_{cq(max)} + t_{su}$$

Therefore

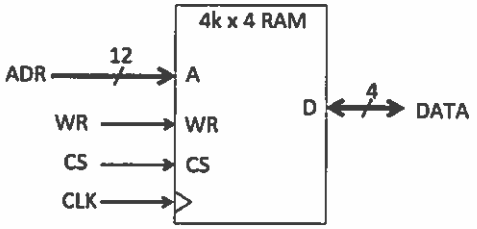
$t_{clk(max)} = 4.2ns$ and hence $f_{clk}(max) = 238.1MHz.$

[1]

Note that hold time violation could be avoided by choosing t_{dly} between 0.1ns and 0.4ns. Above 0.4ns, there will be hold time violation on FF2 D input. If student uses 0.4ns, $t_{clk(max)} = 3.9ns$ and yield a faster $f_{clk}(max)$ of 256.4HHz, give an extra 2 bonus marks for showing evidence of deep understanding.

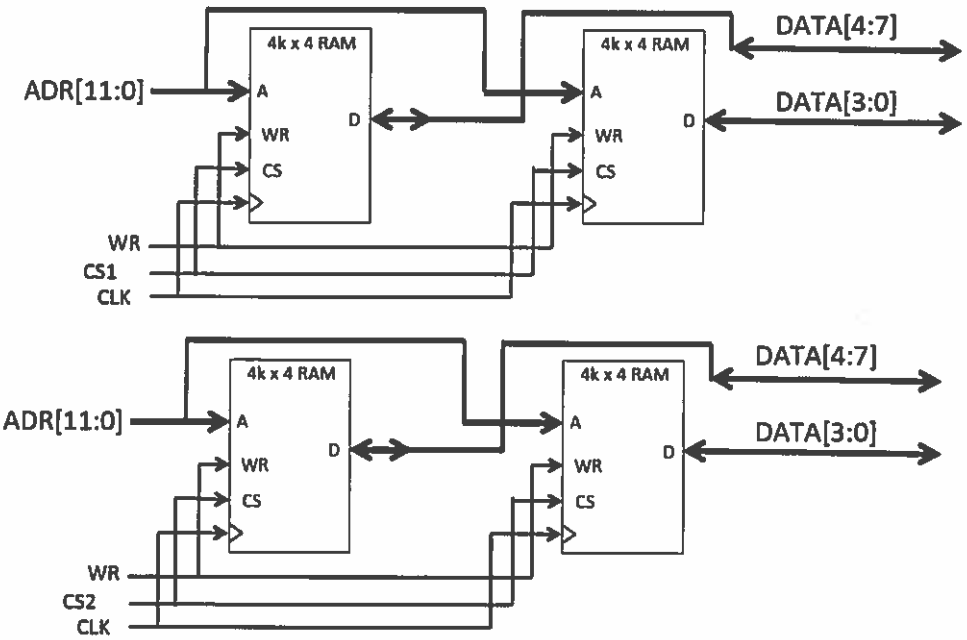
Q1 c) This question tests students' understanding of memory organisation and address decoding.

(i) Address: 12 bits, Data 4 bits



[2]

(ii)



[4]

(iii) CS1 address range: 16'h8000 to 16'h8fff.

CS2 address range: 16'h9000 to 16'h9fff.

Therefore:

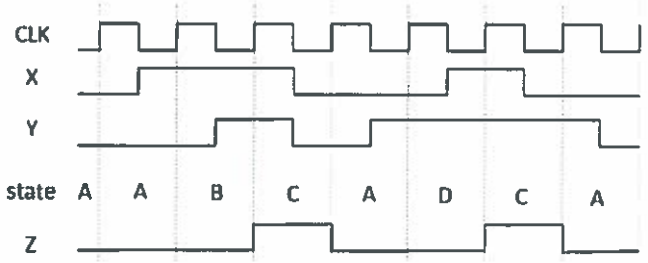
CS1 = A15 & !A14 & !A13 & !A12

CS2 = A15 & !A14 & !A13 & A12

[2]

Q1 (d) This tests students' ability to work with FSM and to specify a simple FSM in Verilog.

(i)



[3]

(ii)

```
module fsm1 (Z, X, Y, CLK);

output   Z;
input    X, Y, CLK;

// define the states - one-hot encoding
parameter S_A = 4'b0001;
parameter S_B = 4'b0010;
parameter S_C = 4'b0100;
parameter S_D = 4'b1000;

reg [3:0]   state;
reg         Z;

wire        CLK, X, Y;

initial     state = 0;

// specify state machine
always @ (posedge CLK) begin
   Z <= 1'b0;     // default Z value
   case (state)
      S_A:  if ((X==1'b1) && (Y==1'b0))
               state <= S_B // XY = 10 -> goto B
            else if ((X==1'b0) && (Y==1'b1))
               state <= S_D // XY = 01 -> goto D
            else if ((X==1'b1) && (Y==1'b1)) begin
               state <= S_C; // XY = 11 -> goto C
               Z <= 1'b1;
               end
            else  state <= S_A; // stay in A

      S_B:  if ((X==1'b0) && (Y==1'b0))
               state <= S_A; // XY = 00 -> goto A
            else if ((X==1'b1) && (Y==1'b1)) begin
               state <= S_C; // XY = 11 -> goto C
               Z <= 1'b1;
               end;
            else  state <= S_B; // stay in B

      S_C:  state <= S_A; // always goto A

      S_D:  if ((X==1'b0) && (Y==1'b0))
               state <= S_A; // XY = 00 -> goto A
            else if ((X==1'b1) && (Y==1'b1)) begin
               state <= S_C; // XY = 11 -> goto C
               Z <= 1'b1;
               end
            else  state <= S_D; // stay in D
      default: state <= S_A;
   endcase
endmodule
```

[5]

Q1 (e) This tests students' understanding of DAC principles, terms and function.

(i) Basic bookwork. The resistor network divides 5V into 1024 voltage values. IN[9:0] is decoder into a one-hot code, which turns ON one of the 1024 switches to route the appropriate voltage from the resistor-divider to the input of the unity gain amplifier. The decoder circuit takes the digital input and set the corresponding output BIT to 1 and the rest to 0 (one-hot code).

[2]

(ii) Vout = 5 x (IN[9:0]+1) / 1025 (volts)

[2]

(iii) Range = 4.89mV to 4.995V, resolution = 4.89mV

[2]

(iv) IN = 10'b0100011010 = 10'h11a. Therefore in decimal IN = 1*256 + 1*16 + 10 = 282.

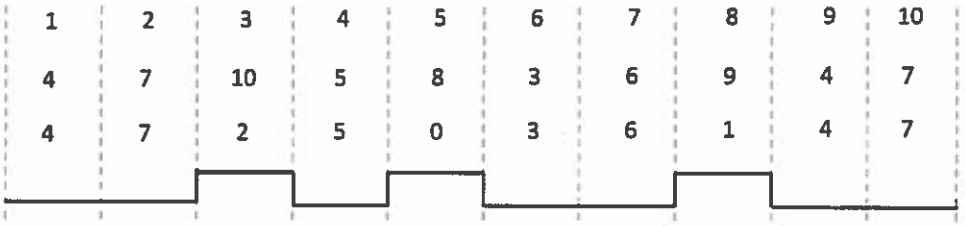
Therefore SW282 is turned ON, and Vout = 5*(282+1)/1025 = 1.38V.

[2]

Q2 This question tests students' ability to walk through the working of a reasonably complex digital circuit, then derive its function. It also tests students' understanding of how circuits could be implemented on FPGAs, and the resources that would be used. Finally it tests students' ability to estimate how fast the circuit might work.

(a) At the end of each clock cycle, Q3:0 = (Q3:0 mod 8) + 3. Hence we get:

Cycle of CLK	1	2	3	4	5	6	7	8	9	10
Q3:0	4	7	10	5	8	3	6	9	4	7
Q2:0	4	7	2	5	0	3	6	1	4	7
Q3										



[10]

(b) Q3:0 (initial) = 0, sequence of Q3:0 is 0, 3, 6, 9, 4, …

Q3:0 (initial) = 1, sequence of Q3:0 is 1, 4, …

Q3:0 (initial) = 2, sequence of Q3:0 is 2, 5, 8, 3, 3, 6, 9, 4, …

Q3:0 (initial) = 3 and 6 are both a subset of Q3:0 = 0

Q3:0 (initial) = 5 is a subset of Q3:0 = 2

Therefore all cases ended up with the same repetitive cycle as Q3:0 = 4.

[5]

(c) Looking at (i), it is clear that Q3:0 repeats itself after 8 cycles and produces 3 pulses on Q3. Therefore average frequency of Q3 is (3/8) x 50MHz = 18.75MHz.

X3 has a frequency one sixteeth of Q3, therefore its average frequency is 18.75/16 = 1.172MHz.

[5]

(d)

M1: 3 LEs (each LE can do one bit adder)

M2: Since each LE also has a D-FF, M2 is part of M1 LEs

M3: Each counter bit requires one LE, therefore 4 LEs

Total LE usage is 7.

[5]

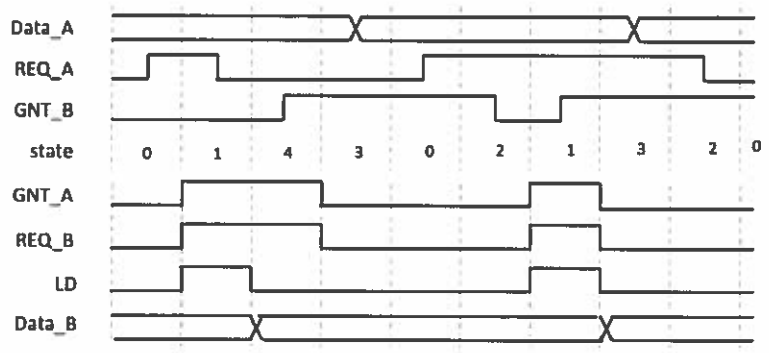
(e)

This circuit has two register stages, both have one level of logic (i.e. only going through one LE). Therefore the worst-case delay is: $t_{cq} + t_{lut} + t_{su} = 0.25 + 0.1 + 0.8 = 1.15$ns. Therefore this circuit can work up to 870MHz. Hold time does not matter.

[5]

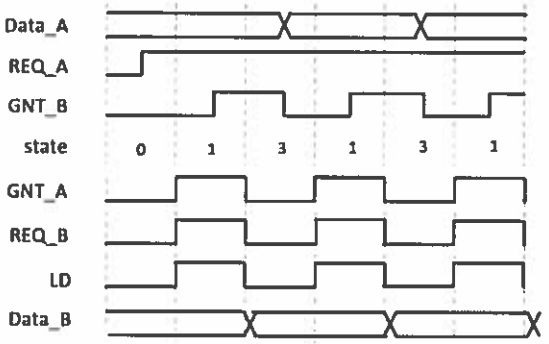
Q3 This question tests students' understanding of interfacing between two systems and how to analyse and design a more complex FSM.

(a)



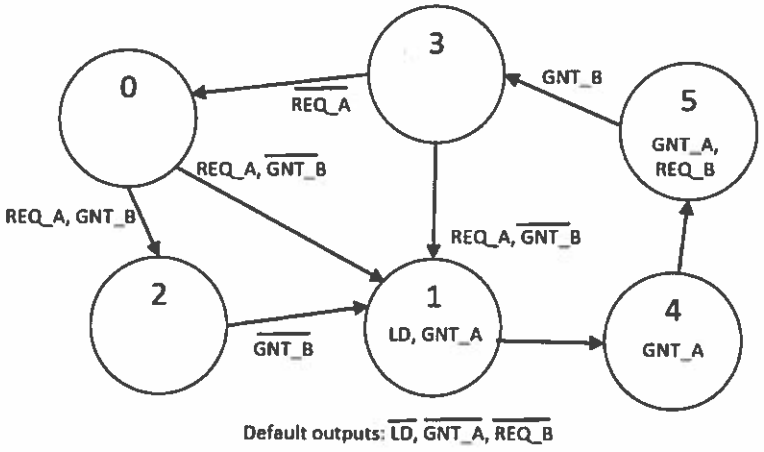
[12]

(b) Assumption is that REQ_A remains high and GNT_B is asserted as soon as REQ_B goes high. Under this circumstance, new data is transferred every two clock cycles.



[10]

(c) To ensure that Data_B is valid for one clock cycle before asserting REQ_B, we need to create a new state as shown below. Data_B becomes valid in state 4, therefore REQ_B cannot be assert until state 5. Now the maximum transfer rate is a new data every 4 clock cycles instead of 2 as before.



[8]